

Version: 1.0



Libft

Your very first own library

Summary

This project is about coding a C library.
It will contain a lot of general purpose functions your programs will rely upon.

#C

#Imperative

#Library

42

Intellectual Property Disclaimer

All content presented in this training module, including but not limited to texts, images, graphics, and other materials, is protected by intellectual property rights held by Association 42.

Terms of Use:

- **Personal use:** You are permitted to use the contents of this module solely for personal purpose. Any commercial use, reproduction, distribution, modification, or public display is strictly prohibited without prior written permission from Association 42.
- **Respect for Integrity:** You must not alter, transform, or adapt the content in any way that could harm its integrity.

Protection of Rights:

Any violation of these terms constitutes an infringement of intellectual property rights and may result in legal action. We reserve the right to take all necessary measures to protect our rights, including but not limited to claims for damages.

For any questions regarding the use of the content or to obtain authorization, please contact:
legal@42.fr

Contents

1	Introduction	1
2	Common Instructions	2
3	AI Instructions	3
4	Mandatory part	5
4.1	Technical considerations	5
4.2	Part 1 - Libc functions	6
4.3	Part 2 - Additional functions	8
4.4	Part 3 - Linked list	12
5	Readme Requirements	15
6	Submission and peer-evaluation	16

Chapter 1

Introduction

C programming can be quite tedious without access to the highly useful standard functions. This project aims to help you understand how these functions work by implementing them yourself and learning to use them effectively. You will create your own library, which will be valuable for your future C school assignments.

Chapter 2

Common Instructions

- Your project must be written in C.
- Your project must be written in accordance with the Norm. If you have bonus files or functions, they are included in the norm check, and you will receive a 0 if there is a norm error inside.
- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc.) except for undefined behaviors. If this happens, your project will be considered non-functional and will receive a 0 during the evaluation.
- All heap-allocated memory space must be properly freed when necessary. No memory leaks will be tolerated.
- If the subject requires it, you must submit a Makefile that will compile your source files to the required output with the flags `-Wall`, `-Wextra`, and `-Werror`, using `cc`, and your Makefile must not relink.
- Your Makefile must at least contain the rules `$(NAME)`, `all`, `clean`, `fclean`, and `re`.
- If your project allows you to use your `libft`, you must copy its sources and its associated Makefile into a `libft` folder. Your project's Makefile must compile the library by using its Makefile, then compile the project.
- We encourage you to create test programs for your project, even though this work **will not be submitted and will not be graded**. It will give you a chance to easily test your work and your peers' work. You will find these tests especially useful during your defense. Indeed, during the defense, you are free to use your tests and/or the tests of the peer you are evaluating.
- Submit your work to your assigned Git repository. Only the work in the Git repository will be graded. If Deepthought is assigned to grade your work, it will be done after your peer evaluations. If an error occurs in any section of your work during Deepthought's grading, the evaluation will stop.

Chapter 3

AI Instructions

● Context

This project is designed to help you discover the fundamental building blocks of your 42 training.

To properly anchor key knowledge and skills, it's essential to adopt a thoughtful approach to using AI tools and support.

True foundational learning requires genuine intellectual effort — through challenge, repetition, and peer-learning exchanges.

For a more complete overview of our stance on AI — as a learning tool, as part of the 42 training, and as an expectation in the job market — please refer to the dedicated FAQ on the intranet.

● Main message

- 👉 Build strong foundations without shortcuts.
- 👉 Really develop tech & power skills.
- 👉 Experience real peer-learning, start learning how to learn and solve new problems.
- 👉 The learning journey is more important than the result.
- 👉 Learn about the risks associated with AI, and develop effective control practices and countermeasures to avoid common pitfalls.

● Learner rules:

- You should apply reasoning to your assigned tasks, especially before turning to AI.
- You should not ask for direct answers to the AI.
- You should learn about 42 global approach on AI.

● Phase outcomes:

Within this foundational phase, you will get the following outcomes:

- Get proper tech and coding foundations.
- Know why and how AI can be dangerous during this phase.

● Comments and example:

- Yes, we know AI exists — and yes, it can solve your projects. But you're here to learn, not to prove that AI has learned. Don't waste your time (or ours) just to demonstrate that AI can solve the given problem.
- Learning at 42 isn't about knowing the answer — it's about developing the ability to find one. AI gives you the answer directly, but that prevents you from building your own reasoning. And reasoning takes time, effort, and involves failure. The path to success is not supposed to be easy.
- Keep in mind that during exams, AI is not available — no internet, no smartphones, etc. You'll quickly realise if you've relied too heavily on AI in your learning process.
- Peer learning exposes you to different ideas and approaches, improving your interpersonal skills and your ability to think divergently. That's far more valuable than just chatting with a bot. So don't be shy — talk, ask questions, and learn together!
- Yes, AI will be part of the curriculum — both as a learning tool and as a topic in itself. You'll even have the chance to build your own AI software. In order to learn more about our crescendo approach you'll go through in the documentation available on the intranet.

✓ Good practice:

I'm stuck on a new concept. I ask someone nearby how they approached it. We talk for 10 minutes — and suddenly it clicks. I get it.

✗ Bad practice:

I secretly use AI, copy some code that looks right. During peer evaluation, I can't explain anything. I fail. During the exam — no AI — I'm stuck again. I fail.

Chapter 4

Mandatory part

Program Name	libft.a
Files to Submit	Makefile, libft.h, ft_*.c
Makefile	NAME, all, clean, fclean, re
External Functions	Detailed below
Libft authorized	n/a
Description	Write your own library: a collection of functions that will be a useful tool for your curriculum.

4.1 Technical considerations

- Declaring global variables is forbidden.
- If you need helper functions to split a more complex function, define them as `static` functions. This way, their scope will be limited to the appropriate file.
- Place all your files at the root of your repository.
- Turning in unused files is forbidden.
- Every `.c` files must compile with the flags `-Wall -Wextra -Werror`.
- You must use the command `ar` to create your library. Using the `libtool` command is forbidden.
- Your `libft.a` has to be created at the root of your repository.

4.2 Part 1 - Libc functions

To begin, you must redo a set of functions from the `libc`. Your functions will have the same prototypes and implement the same behaviors as the originals. They must comply with the way they are defined in their `man` pages. The only difference will be their names. They will begin with the `'ft_'` prefix. For instance, `strlen` becomes `ft_strlen`.



Some of the functions' prototypes you have to redo use the `'restrict'` qualifier. This keyword is part of the `c99` standard. It is therefore forbidden to include it in your own prototypes and to compile your code with the `-std=c99` flag.

You must write your own function implementing the following original ones. They do not require any external functions:



For the character classification functions (`isalpha`, `isdigit`, `isalnum`, `isascii`, `isprint`), the return value must be:

- 1 if the character matches the tested class
- 0 if the character does not match

- | | |
|------------------------|------------------------|
| • <code>isalpha</code> | • <code>toupper</code> |
| • <code>isdigit</code> | • <code>tolower</code> |
| • <code>isalnum</code> | • <code>strchr</code> |
| • <code>isascii</code> | • <code>strrchr</code> |
| • <code>isprint</code> | • <code>strncmp</code> |
| • <code>strlen</code> | • <code>memchr</code> |
| • <code>memset</code> | • <code>memcmp</code> |
| • <code>bzero</code> | • <code>strnstr</code> |
| • <code>memcpy</code> | • <code>atoi</code> |
| • <code>memmove</code> | |
| • <code>strncpy</code> | |
| • <code>strlcat</code> | |

In order to implement the two following functions, you will use `malloc()`:

- `calloc`
- `strdup`



Depending on your current operating system, the `calloc` man page and the function's behavior may differ. The following instruction supersedes what you can find in the man page: If `nmemb` or `size` is 0, then `calloc()` returns a unique pointer value that can later be successfully passed to `free()`.



Some functions that you must reimplement, such as `strncpy`, `strlcat`, and `bzero`, are not included by default in the GNU C Library (glibc). To test them against the system standard, you may need to include `<bsd/string.h>` and compile with the `-lbsd` flag. This behaviour is specific to glibc systems. If you are curious, take the opportunity to explore the differences between glibc and BSD libc.

4.3 Part 2 - Additional functions

In this second part, you must develop a set of functions that are either not in the `libc`, or that are part of it but in a different form.



Some of the functions from part 1 can be useful for implementing the functions below.

Function Name	<code>ft_substr</code>
Prototype	<code>char *ft_substr(char const *s, unsigned int start, size_t len);</code>
Files to Submit	-
Parameters	<code>s</code>: The string from which to create the substring. <code>start</code>: The start index of the substring in the string 's'. <code>len</code>: The maximum length of the substring.
Return Value	The substring. NULL if the allocation fails.
External Functions	<code>malloc</code>
Description	Allocates (with <code>malloc(3)</code>) and returns a substring from the string 's'. The substring begins at index 'start' and is of maximum size 'len'.

Function Name	<code>ft_strjoin</code>
Prototype	<code>char *ft_strjoin(char const *s1, char const *s2);</code>
Files to Submit	-
Parameters	<code>s1</code>: The prefix string. <code>s2</code>: The suffix string.
Return Value	The new string. NULL if the allocation fails.
External Functions	<code>malloc</code>
Description	Allocates (with <code>malloc(3)</code>) and returns a new string, which is the result of the concatenation of 's1' and 's2'.

Function Name	<code>ft_strtrim</code>
Prototype	<code>char *ft_strtrim(char const *s1, char const *set);</code>
Files to Submit	-
Parameters	<code>s1</code>: The string to be trimmed. <code>set</code>: The reference set of characters to trim.
Return Value	The trimmed string. NULL if the allocation fails.
External Functions	<code>malloc</code>
Description	Allocates (with <code>malloc(3)</code>) and returns a copy of 's1' with the characters specified in 'set' removed from the beginning and the end of the string.

Function Name	<code>ft_split</code>
Prototype	<code>char **ft_split(char const *s, char c);</code>
Files to Submit	-
Parameters	<code>s</code>: The string to be split. <code>c</code>: The delimiter character.
Return Value	The array of new strings resulting from the split. NULL if the allocation fails.
External Functions	<code>malloc</code> , <code>free</code>
Description	Allocates (with <code>malloc(3)</code>) and returns an array of strings obtained by splitting 's' using the character 'c' as a delimiter. The array must end with a NULL pointer.

Function Name	<code>ft_itoa</code>
Prototype	<code>char *ft_itoa(int n);</code>
Files to Submit	-
Parameters	<code>n</code> : the integer to convert.
Return Value	The string representing the integer. NULL if the allocation fails.
External Functions	<code>malloc</code>
Description	Allocates (with <code>malloc(3)</code>) and returns a string representing the integer received as an argument. Negative numbers must be handled.

Function Name	<code>ft_strmapi</code>
Prototype	<code>char *ft_strmapi(char const *s, char (*f)(unsigned int, char));</code>
Files to Submit	-
Parameters	s: The string on which to iterate. f: The function to apply to each character.
Return Value	The string created from the successive applications of 'f'. Returns NULL if the allocation fails.
External Functions	<code>malloc</code>
Description	Applies the function 'f' to each character of the string 's', passing its index as the first argument and the character itself as the second. A new string is created (using <code>malloc(3)</code>) to collect the results from the successive applications of 'f'.

Function Name	<code>ft_striteri</code>
Prototype	<code>void ft_striteri(char *s, void (*f)(unsigned int, char*));</code>
Files to Submit	-
Parameters	s: The string on which to iterate. f: The function to apply to each character.
Return Value	None
External Functions	None
Description	Applies the function 'f' on each character of the string passed as argument, passing its index as first argument. Each character is passed by address to 'f' to be modified if necessary.

Function Name	<code>ft_putchar_fd</code>
Prototype	<code>void ft_putchar_fd(char c, int fd);</code>
Files to Submit	-
Parameters	c: The character to output. fd: The file descriptor on which to write.
Return Value	None
External Functions	<code>write</code>
Description	Outputs the character 'c' to the given file descriptor.

Function Name	<code>ft_putstr_fd</code>
Prototype	<code>void ft_putstr_fd(char *s, int fd);</code>
Files to Submit	-
Parameters	s: The string to output. fd: The file descriptor on which to write.
Return Value	None
External Functions	write
Description	Outputs the string 's' to the given file descriptor.

Function Name	<code>ft_putendl_fd</code>
Prototype	<code>void ft_putendl_fd(char *s, int fd);</code>
Files to Submit	-
Parameters	s: The string to output. fd: The file descriptor on which to write.
Return Value	None
External Functions	write
Description	Outputs the string 's' to the given file descriptor followed by a newline.

Function Name	<code>ft_putnbr_fd</code>
Prototype	<code>void ft_putnbr_fd(int n, int fd);</code>
Files to Submit	-
Parameters	n: The integer to output. fd: The file descriptor on which to write.
Return Value	None
External Functions	write
Description	Outputs the integer 'n' to the given file descriptor.

4.4 Part 3 - Linked list

In the third part, you will have to implement functions using a structure in order to manipulate linked lists.

To do so, here is the structure declaration that you should add to your `libft.h` file:

```
ft_list.h

typedef struct      s_list
{
    void            *content;
    struct s_list   *next;
}                  t_list;
```

The members of the `t_list` struct are:

- `content`: The data contained in the node.
Using `void *` allows you to store any type of data.
- `next`: The address of the next node, or `NULL` if the next node is the last one.

Implement the following functions in order to easily use your lists.

Function Name	<code>ft_lstnew</code>
Prototype	<code>t_list *ft_lstnew(void *content);</code>
Files to Submit	-
Parameters	<code>content</code> : The content to store in the new node.
Return Value	A pointer to the new node
External Functions	<code>malloc</code>
Description	Allocates memory (using <code>malloc(3)</code>) and returns a new node. The 'content' member variable is initialized with the given parameter 'content'. The variable 'next' is initialized to <code>NULL</code> .

Function Name	<code>ft_lstadd_front</code>
Prototype	<code>void ft_lstadd_front(t_list **lst, t_list *new);</code>
Files to Submit	-
Parameters	<code>lst</code> : The address of a pointer to the first node of a list. <code>new</code> : The address of a pointer to the node to be added.
Return Value	None
External Functions	None
Description	Adds the node 'new' at the beginning of the list.

Function Name	<code>ft_lstsize</code>
Prototype	<code>int ft_lstsize(t_list *lst);</code>
Files to Submit	-
Parameters	<code>lst</code> : The beginning of the list.
Return Value	The length of the list
External Functions	None
Description	Counts the number of nodes in the list.

Function Name	<code>ft_lstlast</code>
Prototype	<code>t_list *ft_lstlast(t_list *lst);</code>
Files to Submit	-
Parameters	<code>lst</code> : The beginning of the list.
Return Value	Last node of the list
External Functions	None
Description	Returns the last node of the list.

Function Name	<code>ft_lstadd_back</code>
Prototype	<code>void ft_lstadd_back(t_list **lst, t_list *new);</code>
Files to Submit	-
Parameters	<code>lst</code> : The address of a pointer to the first node of a list. <code>new</code> : The address of a pointer to the node to be added.
Return Value	None
External Functions	None
Description	Adds the node 'new' at the end of the list.

Function Name	<code>ft_lstdelone</code>
Prototype	<code>void ft_lstdelone(t_list *lst, void (*del)(void *));</code>
Files to Submit	-
Parameters	<code>lst</code> : The node to free. <code>del</code> : The address of the function used to delete the content.
Return Value	None
External Functions	<code>free</code>
Description	Takes a node as parameter and frees its content using the function 'del'. Frees the node itself but does NOT free the next node.

Function Name	<code>ft_lstclear</code>
Prototype	<code>void ft_lstclear(t_list **lst, void (*del)(void *));</code>
Files to Submit	-
Parameters	<code>lst</code>: The address of a pointer to a node. <code>del</code>: The address of the function used to delete the content of the node.
Return Value	None
External Functions	<code>free</code>
Description	Deletes and frees the given node and all its successors, using the function 'del' and <code>free(3)</code> . Finally, set the pointer to the list to NULL.

Function Name	<code>ft_lstiter</code>
Prototype	<code>void ft_lstiter(t_list *lst, void (*f)(void *));</code>
Files to Submit	-
Parameters	<code>lst</code>: The address of a pointer to a node. <code>f</code>: The address of the function to apply to each node's content.
Return Value	None
External Functions	None
Description	Iterates through the list 'lst' and applies the function 'f' to the content of each node.

Function Name	<code>ft_lstmap</code>
Prototype	<code>t_list *ft_lstmap(t_list *lst, void *(*f)(void *), void (*del)(void *));</code>
Files to Submit	-
Parameters	<code>lst</code>: The address of a pointer to a node. <code>f</code>: The address of the function applied to each node's content. <code>del</code>: The address of the function used to delete a node's content if needed.
Return Value	The new list. NULL if the allocation fails.
External Functions	<code>malloc, free</code>
Description	Iterates through the list 'lst', applies the function 'f' to each node's content, and creates a new list resulting of the successive applications of the function 'f'. The 'del' function is used to delete the content of a node if needed.

Chapter 5

Readme Requirements

A README.md file must be provided at the root of your Git repository. Its purpose is to allow anyone unfamiliar with the project (peers, staff, recruiters, etc.) to quickly understand what the project is about, how to run it, and where to find more information on the topic.

The README.md must include at least:

- The very first line must be italicized and read: *This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]]*.
- A “**Description**” section that clearly presents the project, including its goal and a brief overview.
- An “**Instructions**” section containing any relevant information about compilation, installation, and/or execution.
- A “**Resources**” section listing classic references related to the topic (documentation, articles, tutorials, etc.), as well as a description of how AI was used — specifying for which tasks and which parts of the project.

➡ **Additional sections may be required depending on the project** (e.g., usage examples, feature list, technical choices, etc.).

Any required additions will be explicitly listed below.

- A detailed description of the library created for this project must also be included.



The choice of language is at your discretion. It is recommended to write in English, but it is not mandatory.

Chapter 6

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the peer-evaluation. Don't hesitate to double check the names of your files to ensure they are correct.



Place all your files at the root of your repository.

During the evaluation, a brief **modification of the project** may occasionally be requested. This could involve a minor behavior change, a few lines of code to write or rewrite, or an easy-to-add feature.

While this step may **not be applicable to every project**, you must be prepared for it if it is mentioned in the evaluation guidelines.

This step is meant to verify your actual understanding of a specific part of the project. The modification can be performed in any development environment you choose (e.g., your usual setup), and it should be feasible within a few minutes — unless a specific timeframe is defined as part of the evaluation.

You can, for example, be asked to make a small update to a function or script, modify a display, or adjust a data structure to store new information, etc.

The details (scope, target, etc.) will be specified in the **evaluation guidelines** and may vary from one evaluation to another for the same project.